

# Bank Customer Churn Prediction Using Explainable Machine Learning and Interactive Decision Support System

Himanshu Gupta

ML Engineer | Final Year B.Tech CSE

IEEE-style research documentation generated from repository inspection

*This paper is based on the implementation present in the local repository. It does not claim unimplemented XGBoost training, cross-validation, hyperparameter search, or executed SHAP artifacts.*

# **Table of Contents**

Abstract

I. Introduction

II. Literature Review

III. Problem Statement

IV. Dataset Description

V. Methodology

VI. System Architecture

VII. Model Development

VIII. Explainable AI Framework

IX. Risk Scoring Framework

X. What-If Simulation Engine

XI. Experimental Results

XII. Deployment and Production Readiness

XIII. Discussion

XIV. Future Work

XV. Conclusion

References

**\*\*Author:\*\*** Himanshu Gupta  
**\*\*Project:\*\*** Bank Customer Churn Prediction and Risk Scoring System  
**\*\*Repository inspected:\*\*** `D:\bank-churn-predictor`  
**\*\*Dataset inspected:\*\*** `data\europaean\_bank.csv`

## Abstract

Customer churn prediction is a critical analytical task for retail banks because undetected customer attrition can reduce deposit stability, product adoption, and long-term customer value. This paper presents an implementation-driven study of a bank customer churn prediction system that combines supervised machine learning, explainability-oriented feature reporting, and an interactive Streamlit decision support interface. The implemented repository uses a European bank customer dataset containing 10000 records and 14 columns. The pipeline validates required columns, removes direct identifiers, imputes missing values, one-hot encodes geography and gender, standardizes configured numeric attributes, and creates four derived behavioral features. Three scikit-learn classifiers are implemented: Logistic Regression, Random Forest, and Gradient Boosting. A stratified 80:20 train-test split is used, and model selection is based on ROC-AUC. In a reproduced holdout evaluation following the repository logic, Gradient Boosting achieved an accuracy of 0.8685, precision of 0.7857, recall of 0.4865, F1-score of 0.6009, and ROC-AUC of 0.8704. The deployed application exposes a churn risk calculator, a feature-importance dashboard, and a what-if simulator. The system persists the model, scaler, encoder, and feature order for consistent inference and includes Docker-based deployment support. The study also identifies implementation limitations: XGBoost is listed as a dependency but is not instantiated in the training code, SHAP computation exists as a utility but is not called by the training pipeline, and no cross-validation or hyperparameter search is implemented.

**\*\*Keywords:\*\*** customer churn prediction, banking analytics, gradient boosting, explainable machine learning, SHAP, Streamlit, Docker, risk scoring.

## I. Introduction

Customer churn describes the condition in which an existing customer terminates or materially reduces a relationship with a service provider. In banking, this problem is operationally important because customer acquisition costs, regulatory onboarding effort, and relationship-building activities are generally higher than the cost of targeted retention. A churn prediction system can support earlier intervention by ranking customers according to predicted attrition probability.

The inspected project implements a practical churn prediction and risk scoring workflow. It is not limited to a notebook-style experiment; it includes modular Python source code, persisted preprocessing artifacts, a saved model, an interactive Streamlit dashboard, and Docker deployment configuration. The system is designed for analysts or bank retention teams who need an interface for individual customer risk scoring and exploratory what-if analysis.

The motivation of this paper is to document the implemented system in a reproducible IEEE-style form. All architectural claims, preprocessing steps, metrics, risk thresholds, and deployment details are taken from the repository. Where the repository documentation and code differ, the code is treated as authoritative.

## II. Literature Review

Customer churn prediction has been studied across banking, telecommunications, subscription services, and digital platforms. Recent work continues to compare classical machine learning methods, ensemble learners, neural models, and explainable artificial intelligence techniques. In banking-related churn and loyalty studies, tree-based ensembles such as Gradient Boosting, Random Forest, and XGBoost are frequently evaluated because customer churn relationships are nonlinear and may involve interaction effects among demographics, activity, balance, and product usage.

Zhang et al. investigated bank customer loyalty prediction using XGBoost and compared multiple machine learning models for identifying potential churn-oriented loyalty risk. Liu et al. proposed a hybrid neural network churn prediction model and noted that traditional algorithms such as Logistic Regression, Decision Trees, Random Forests, LightGBM, AdaBoost, and XGBoost remain common baselines. Studies on interpretable churn prediction also emphasize that high predictive performance alone is insufficient for business adoption; retention teams require explanations that identify which variables contribute to risk. SHAP, introduced by Lundberg and Lee, provides an additive feature attribution framework based on Shapley values and has become widely used for both global and local explanations.

Compared with these approaches, the inspected project is narrower but operationally complete. It implements a tabular supervised-learning pipeline and dashboard but does not implement deep learning, oversampling, temporal modeling, cost-sensitive optimization, or hyperparameter search. It also includes an XGBoost dependency in `requirements.txt`, but the training source code does not instantiate an XGBoost classifier. Therefore, XGBoost is discussed as relevant literature and as a future extension, not as an implemented experimental result.

## III. Problem Statement

The business challenge is to identify bank customers likely to exit before churn occurs, assign an interpretable risk label, and provide a lightweight decision support interface for analysts. Formally, given a customer feature vector  $x$ , the objective is to estimate:

$P(y = 1 | x)$ ,

where  $y = 1$  represents churn as encoded by the target column `Exited`.

The implemented objectives are:

1. transform raw customer attributes into a consistent model-ready representation;
2. train and compare supervised classification models;
3. persist a selected model and preprocessing artifacts;
4. expose a single prediction interface for the application layer;
5. support risk interpretation using probability thresholds and feature-importance artifacts.

## IV. Dataset Description

The inspected dataset is stored at `data\europaan\_bank.csv` and contains 10000 observations and 14 columns. The raw columns are: `Year, CustomerId, Surname, CreditScore, Geography, Gender, Age, Tenure, Balance, NumOfProducts, HasCrCard, IsActiveMember, EstimatedSalary, Exited`. The implemented training code uses `CreditScore, Age, Tenure, Balance, NumOfProducts, HasCrCard, IsActiveMember, EstimatedSalary, Geography, Gender, Exited`. It drops `CustomerId, Surname` and ignores `Year` during model preparation.

**\*\*Table I. Dataset schema used by the implementation\*\***

| Column          | Role in implementation   |
|-----------------|--------------------------|
| CreditScore     | Numeric predictor        |
| Age             | Numeric predictor        |
| Tenure          | Numeric predictor        |
| Balance         | Numeric predictor        |
| NumOfProducts   | Numeric predictor        |
| HasCrCard       | Numeric binary predictor |
| IsActiveMember  | Numeric binary predictor |
| EstimatedSalary | Numeric predictor        |
| Geography       | Categorical predictor    |
| Gender          | Categorical predictor    |
| Exited          | Target variable          |

**\*\*Table II. Numeric feature summary\*\***

| Feature         | Mean       | Std. Dev. | Min     | Median     | Max        |
|-----------------|------------|-----------|---------|------------|------------|
| CreditScore     | 650.529    | 96.653    | 350.000 | 652.000    | 850.000    |
| Age             | 38.922     | 10.488    | 18.000  | 37.000     | 92.000     |
| Tenure          | 5.013      | 2.892     | 0.000   | 5.000      | 10.000     |
| Balance         | 76485.889  | 62397.405 | 0.000   | 97198.540  | 250898.090 |
| NumOfProducts   | 1.530      | 0.582     | 1.000   | 1.000      | 4.000      |
| HasCrCard       | 0.706      | 0.456     | 0.000   | 1.000      | 1.000      |
| IsActiveMember  | 0.515      | 0.500     | 0.000   | 1.000      | 1.000      |
| EstimatedSalary | 100090.240 | 57510.493 | 11.580  | 100193.915 | 199992.480 |

The target distribution is imbalanced: class 0 contains 7963 customers (79.63%), while class 1 contains 2037 customers (20.37%). No missing values are present in the inspected CSV; however, the implemented pipeline includes median and mode imputation for future datasets with missing values.

## V. Methodology

### A. Data Preprocessing

The preprocessing module performs four operations. First, `load\_data(path)` reads the CSV file and validates that all configured columns exist. Second, it removes `CustomerId` and `Surname`. Third, `handle\_missing\_values(df)` logs missing-value counts and fills numeric nulls with medians and categorical nulls with modes. Fourth, categorical and numeric transformations are persisted for inference.

Categorical variables `Geography, Gender` are one-hot encoded with `drop='first'` and `handle\_unknown='ignore'`. The learned encoded columns are `Geography\_Germany, Geography\_Spain, Gender\_Male`. Numeric variables are transformed with `StandardScaler`, following:

$$z = (x - \mu) / \sigma$$

### B. Feature Engineering

The implementation adds four derived features:

1. `balance\_salary\_ratio = Balance / (EstimatedSalary + 1)`;
2. `age\_tenure\_interaction = Age \* Tenure`;
3. `product\_engagement = NumOfProducts \* IsActiveMember`;
4. `zero\_balance\_flag = 1 if Balance = 0 else 0`.

The final model feature count is 15. The persisted feature order is:

`CreditScore, Age, Tenure, Balance, NumOfProducts, HasCrCard, IsActiveMember, EstimatedSalary, balance\_salary\_ratio, age\_tenure\_interaction, product\_engagement, zero\_balance\_flag, Geography\_Germany, Geography\_Spain, Gender\_Male`.

### C. Model Selection Rationale

The repository compares a linear baseline, a bagging-based tree ensemble, and a boosting-based tree ensemble. Logistic Regression

provides a simple linear benchmark; Random Forest provides nonlinear bagging with class weighting; Gradient Boosting provides sequential additive modeling. The implemented model-selection criterion is ROC-AUC on a stratified holdout test split.

## VI. System Architecture

The architecture separates configuration, preprocessing, training, evaluation, prediction, and application layers.

**Figure 1. System architecture diagram**

```
flowchart LR
    A[CSV dataset] --> B[src.data_preprocessing]
    B --> C[src.feature_engineering]
    C --> D[src.model_training]
    D --> E[models/best_model.pkl]
    D --> F[artifacts/scaler.pkl encoder.pkl feature_names.pkl]
    D --> G[artifacts/shap_importance.csv]
    E --> H[src.predict.predict_churn]
    F --> H
    G --> H
    H --> I[Streamlit app.py]
    I --> J[Risk calculator Feature importance What-if simulator]
```

**Figure 2. Machine learning pipeline diagram**

```
flowchart TD
    A[Load data] --> B[Validate required columns]
    B --> C[Drop CustomerId and Surname]
    C --> D[Median/mode imputation]
    D --> E[Create engineered features]
    E --> F[Stratified train-test split]
    F --> G[Fit encoder and scaler on train data]
    G --> H[Train candidate classifiers]
    H --> I[Evaluate metrics]
    I --> J[Select highest ROC-AUC model]
    J --> K[Persist model and artifacts]
```

The prediction pipeline loads the persisted model, scaler, encoder, and feature order. A raw input dictionary is converted to a single-row dataframe, feature engineering is applied, categorical features are encoded, numeric features are scaled, columns are reindexed to the saved order, and `predict\_proba` is used to compute churn probability.

## VII. Model Development

The implemented candidate models and parameters are:

**Table III. Implemented models**

| Model               | Important implemented settings                                                                 |
|---------------------|------------------------------------------------------------------------------------------------|
| Logistic Regression | `max_iter=1000`, `random_state=42`, L2 penalty through scikit-learn default                    |
| Random Forest       | `n_estimators=100`, `class_weight='balanced'`, `random_state=42`, `n_jobs=1`                   |
| Gradient Boosting   | `random_state=42`, scikit-learn default `n_estimators=100`, `learning_rate=0.1`, `max_depth=3` |

No hyperparameter search and no cross-validation strategy are implemented in the repository. The split is a stratified 80:20 holdout controlled by `TEST\_SIZE=0.2` and `RANDOM\_STATE=42`.

### XGBoost Status and Objective Function

The dependency list includes `xgboost`, and the README describes XGBoost as part of the intended technology stack. However, `src/model\_training.py` does not import or instantiate XGBoost. Consequently, this paper does not report XGBoost metrics as project results.

For reference, the regularized XGBoost objective commonly used in the literature is:

$$\text{Obj} = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k),$$

where:

$$\Omega(f) = \gamma T + (1/2) \lambda \sum_j w_j^2.$$

This formulation is included only as methodological background and a future implementation direction.

## VIII. Explainable AI Framework

The repository contains `compute\_shap\_values(model, X\_test, feature\_names)` in `src/model\_evaluation.py`. This function uses `shap.Explainer`, creates a SHAP summary bar plot, and writes `artifacts/shap\_importance.csv`. The training pipeline, however, does not call this SHAP function. Instead, `src/model\_training.py` writes a feature-importance CSV using `feature\_importances\_` for tree models, absolute coefficients for linear models, or zeros otherwise.

SHAP explanations are based on the additive explanation model:

$$g(z) = \phi_0 + \sum_i \phi_i z_i,$$

where  $\phi_i$  is the attribution assigned to feature `i`. A global importance score is commonly computed as:

$$I_j = (1/n) * \sum_i |\phi_j^{(i)}|$$

In the current artifact used by the dashboard, the stored importance values are model-native importances from the selected Gradient Boosting model, not verified SHAP values from an executed SHAP computation.

**\*\*Table IV. Top reproduced feature importances\*\***

| Feature              | Importance |
|----------------------|------------|
| Age                  | 0.3867     |
| NumOfProducts        | 0.2839     |
| product_engagement   | 0.1032     |
| Balance              | 0.0629     |
| Geography_Germany    | 0.0587     |
| balance_salary_ratio | 0.0247     |
| IsActiveMember       | 0.0217     |
| CreditScore          | 0.0171     |
| EstimatedSalary      | 0.0134     |
| Gender_Male          | 0.0133     |

## IX. Risk Scoring Framework

The prediction module returns three values: `churn_probability`, `churn_flag`, and `risk_label`. The churn flag is computed using the configured threshold:

`churn_flag = 1 if P(churn) >= 0.5 else 0`.

The risk bands are:

**\*\*Table V. Risk scoring thresholds\*\***

| Risk label | Probability range                    | Business interpretation                                                   |
|------------|--------------------------------------|---------------------------------------------------------------------------|
| Low        | <code>p &lt; 0.30</code>             | Customer appears comparatively stable; standard engagement is sufficient. |
| Medium     | <code>0.30 &lt;= p &lt;= 0.60</code> | Customer warrants monitoring and targeted retention outreach.             |
| High       | <code>p &gt; 0.60</code>             | Customer should be prioritized for immediate retention review.            |

These labels are deterministic application-layer mappings from probability to decision categories.

## X. What-If Simulation Engine

The Streamlit application implements a what-if simulator in the third tab. It reuses the same input collection pattern as the risk calculator, calls `predict_churn` automatically on input changes, and stores a probability history in `st.session_state`. The trend chart shows how probability changes as the user adjusts customer attributes.

The simulator is not a causal inference engine. It does not estimate treatment effects or recommend optimal interventions. Its value is operational: it exposes sensitivity of the trained predictive model to changes in input attributes and helps analysts reason about possible retention scenarios.

## XI. Experimental Results

The reproduced experiment follows the repository logic with a compatibility adjustment for the local scikit-learn version. The implemented code uses `sparse_output=False`, which is valid in newer scikit-learn versions; the local runtime uses an older scikit-learn release, so the reproduction uses the equivalent `sparse=False` argument. No model hyperparameters were changed.

**\*\*Table VI. Holdout model comparison\*\***

| Model               | Accuracy | Precision | Recall | F1-score | ROC-AUC | Confusion matrix         |
|---------------------|----------|-----------|--------|----------|---------|--------------------------|
| gradient_boosting   | 0.8685   | 0.7857    | 0.4865 | 0.6009   | 0.8704  | [[1539, 54], [209, 198]] |
| random_forest       | 0.8590   | 0.7706    | 0.4373 | 0.5580   | 0.8539  | [[1540, 53], [229, 178]] |
| logistic_regression | 0.8090   | 0.5954    | 0.1916 | 0.2900   | 0.7734  | [[1540, 53], [329, 78]]  |

The highest ROC-AUC is obtained by Gradient Boosting. Its confusion matrix is:

`[[1539, 54], [209, 198]]`.

This corresponds to 1539 true negatives, 54 false positives, 209 false negatives, and 198 true positives at the configured threshold of 0.5. The model obtains high specificity but moderate recall, indicating that it misses a nontrivial number of churners. In a production banking setting, the threshold should therefore be calibrated against campaign cost and churn-loss economics.

## XII. Deployment and Production Readiness

The application layer is implemented in `app.py` using Streamlit and Plotly. It contains three tabs: Churn Risk Calculator, Feature Importance, and What-If Simulator. The application imports `predict_churn` from `src.predict` and delegates preprocessing and inference to that module.

The Dockerfile uses `python:3.10-slim`, sets `/app` as the working directory, copies `requirements.txt` before project files to improve layer caching, installs dependencies with `--no-cache-dir`, creates a non-root `appuser`, exposes port 8501, and defines a `/healthz` health check. The command starts Streamlit on `0.0.0.0:8501` with headless mode and usage statistics disabled.

**\*\*Figure 3. Deployment architecture diagram\*\***

flowchart LR

```

A[Source repository] --> B[Docker build]
B --> C[Python 3.10 slim image]

```

```
C --> D[Installed requirements]
D --> E[Copied application files]
E --> F[Non-root Streamlit process]
F --> G[Port 8501]
G --> H[/healthz health check]
```

Production readiness strengths include modular source separation, artifact persistence, centralized configuration, Dockerization, health checks, and a single inference interface. Remaining production gaps include unpinned dependencies in the inspected `requirements.txt`, absence of automated tests, absence of CI/CD, no model registry, no input schema validation beyond required fields, and no monitoring for data drift or prediction drift.

### XIII. Discussion

The project demonstrates a complete baseline workflow for bank churn prediction. Its strongest engineering qualities are modularity, artifact persistence, consistent inference transformation, dashboard usability, and explicit risk thresholds. The implemented derived features also encode business-relevant interactions such as balance relative to salary and product engagement.

The main limitations are implementation-level rather than conceptual. First, XGBoost is not implemented despite being listed in documentation and dependencies. Second, SHAP utilities exist but the current training pipeline writes model-native importances rather than executing SHAP. Third, the evaluation uses a single stratified holdout split without cross-validation or hyperparameter tuning. Fourth, the model threshold is fixed at 0.5 and is not optimized for retention economics. Fifth, the Docker image excludes CSV files through `.dockerignore`, which is suitable for inference if artifacts are present but prevents retraining inside the image without mounting data.

The business impact of the current system is best framed as decision support rather than automated decision-making. The output can help prioritize retention review, but operational use would require governance, threshold calibration, monitoring, and periodic retraining.

### XIV. Future Work

Future work should focus on:

1. implementing XGBoost and comparing it with the current scikit-learn models;
2. adding cross-validation and hyperparameter optimization;
3. executing SHAP computation as part of training and storing verified SHAP artifacts;
4. adding MLflow or a similar model registry for experiment tracking and artifact versioning;
5. creating CI/CD checks for syntax, unit tests, training smoke tests, and Docker builds;
6. adding probability calibration and threshold optimization based on retention cost;
7. implementing drift monitoring for input distributions and prediction distributions;
8. supporting batch scoring and authenticated deployment for real banking workflows.

### XV. Conclusion

This paper documented an implemented bank churn prediction and decision support system through repository inspection and reproduced evaluation. The project uses structured preprocessing, engineered features, three supervised classifiers, persisted artifacts, a Streamlit dashboard, and Docker deployment support. The reproduced holdout evaluation selected Gradient Boosting as the strongest implemented model by ROC-AUC, with ROC-AUC of 0.8704. The system provides useful churn probability scoring and interactive analysis, but it should be interpreted as a strong baseline rather than a fully governed production ML platform. The most important next steps are implementing the documented XGBoost extension, integrating verified SHAP explanations into the training pipeline, adding cross-validation and hyperparameter tuning, and introducing MLOps controls for deployment, monitoring, and retraining.

### References

- [1] B. Ranga, "Bank Customer Churn Dataset," Kaggle, 2023. Available: <https://www.kaggle.com/datasets/bhuviranga/customer-churn-data>
- [2] M. Zangari, "Bank Customer Churn Prediction," Kaggle, 2024. Available: <https://www.kaggle.com/datasets/murilozangari/customer-churn-from-a-bank>
- [3] Y. Zhang, K. Chen, and T. Chen, "Analysis and Prediction of Bank Customer Loyalty Based on XGBoost Algorithm," \*Advances in Artificial Intelligence, Big Data and Algorithms\*, IOS Press, 2023, doi: 10.3233/FAIA230865.
- [4] X. Liu et al., "Customer churn prediction model based on hybrid neural networks," \*Scientific Reports\*, vol. 14, no. 30707, 2024.
- [5] A. De Caigny, K. Coussement, and K. W. De Bock, "Hybrid black-box classification for customer churn prediction with segmented interpretability analysis," \*Decision Support Systems\*, vol. 181, 2024.
- [6] S. Prabu and collaborators, "Predicting customer churn: A systematic literature review," \*Journal of Discrete Mathematical Sciences and Cryptography\*, vol. 25, no. 7, 2022.
- [7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in \*Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining\*, 2016, pp. 785-794.

- [8] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *\*The Annals of Statistics\**, vol. 29, no. 5, pp. 1189-1232, 2001.
- [9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *\*Advances in Neural Information Processing Systems\**, vol. 30, 2017.
- [10] L. Breiman, "Random forests," *\*Machine Learning\**, vol. 45, pp. 5-32, 2001.
- [11] D. R. Cox, "The regression analysis of binary sequences," *\*Journal of the Royal Statistical Society: Series B\**, vol. 20, no. 2, pp. 215-242, 1958.
- [12] T. Fawcett, "An introduction to ROC analysis," *\*Pattern Recognition Letters\**, vol. 27, no. 8, pp. 861-874, 2006.
- [13] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *\*Journal of Machine Learning Research\**, vol. 12, pp. 2825-2830, 2011.
- [14] scikit-learn Developers, "GradientBoostingClassifier," scikit-learn documentation. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html>
- [15] scikit-learn Developers, "OneHotEncoder," scikit-learn documentation. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html>
- [16] Streamlit Inc., "st.plotly\_chart," Streamlit documentation. Available: [https://docs.streamlit.io/develop/api-reference/charts/st.plotly\\_chart](https://docs.streamlit.io/develop/api-reference/charts/st.plotly_chart)
- [17] Docker Inc., "Dockerfile reference," Docker documentation. Available: <https://docs.docker.com/reference/builder/>
- [18] Docker Inc., "Building best practices," Docker documentation. Available: <https://docs.docker.com/build/building/best-practices/>